

Практична робота № 1 Асимптотична складність алгоритмів.

О-нотація

13. Задано функції $f(n) = 100n^3 + 8$ та $g(n) = n^3$. Доведіть, що $f(n) = O(g(n))$.

$f(n) \in O(g(n))$ якщо існують сталі $c > 0$ і n_0 такі, що $f(n) \leq c \cdot g(n)$ для $\forall n \geq n_0$

для $\forall n \geq 1$ маємо $n^3 \geq 1$, отже $8 \leq 8n^3$

Тому для $n \geq 1$

$$f(n) = 100n^3 + 8 \leq 100n^3 + 8n^3 = 108n^3$$

Отже можна вибрати $c = 108$ та $n_0 = 1$. Тоді для всіх $n \geq n_0$

$$f(n) \leq c \cdot g(n)$$

$$100n^3 + 8 \in O(n^3).$$

3. Дано функцію $f(n) = \log(n) + 2n^2 + 11$. Знайти асимптотичну складність у О-нотації.

$f(n) \in O(g(n))$ для $g(n) = n^2$, якщо існують сталі $c > 0$ і n_0 такі, що

$$f(n) \leq c \cdot n^2 \text{ для всіх } n \geq n_0$$

Для $\forall n \geq 2$ виконується $\log(n) \leq n \leq n^2$, отже для $n \geq 2$, $\log(n) \leq n^2$

Тому при $n \geq 2$

$$f(n) = \log(n) + 2n^2 + 11 \leq n^2 + 2n^2 + 11 = 3n^2 + 11$$

Для $n \geq 2$ маємо $3n^2 \geq 12$, отже $11 \leq 3n^2$. Звідси

$$3n^2 + 11 \leq 3n^2 + 3n^2 = 6n^2$$

Отже для всіх $n \geq n_0 = 2$

$$f(n) \leq 6n^2$$

$c = 6$ і $n_0 = 2$ задовольняє означення, тому

$$\log(n) + 2n^2 + 11 \in O(n^2).$$

Контрольні питання

1. Що таке асимптотична складність алгоритму?

Асимптотична складність алгоритму — це спосіб оцінювання його продуктивності при збільшенні розміру вхідних даних до нескінченності, який показує, як швидко алгоритм працює, коли задача стає великою.

2. Яким чином визначається *O*-нотація і яка її сутність?

Сутність *O*-нотації:

Оцінка складності: *O*-нотація дає приблизну оцінку продуктивності алгоритму, що є критично важливим при аналізі великих наборів даних.

Зростання у найгіршому випадку: Вона фокусується на найгіршому сценарії, показуючи, як час роботи або використані ресурси зростають зі збільшенням вхідних даних.

Абстракція від деталей: Нотація дозволяє абстрагуватися від конкретних деталей реалізації, апаратного забезпечення та точних констант, зосереджуючись на поведінці алгоритму при зростанні обсягу даних.

Порівняння алгоритмів: Це стандартизований спосіб порівнювати ефективність різних алгоритмів, розуміючи, який з них буде працювати краще для великих вхідних даних.

Визначення: Нехай f і g — дві функції, що відображають натуральні у простір дійсних чисел. Тоді $f(n) = O(g(n))$ (читається як « f є O велике від g »), якщо $\exists$ дійсна константа $c > 0$ і натуральна константа n_0 , така що $f(n) \leq cg(n)$, для $\forall n \geq n_0$. Ми говоримо, що f зростає щонайбільше так швидко, як g , або не швидше, ніж g . Мета умови $n \geq n_0$ полягає в тому, щоб ігнорувати поведінку f порівняно з g , коли n мало.

3. Які основні правила використання O-нотації при аналізі алгоритмів?

- 1) Ігнорувати константи та нижчі порядки члена: ми дивимося на зростання функції при великих значеннях вхідних даних, тому постійні множники та члени меншого порядку не мають значення для верхньої межі складності.
- 2) Оцінювати найгірший випадок: O-нотація описує верхню межу роботи алгоритму, тому ми аналізуємо сценарій, коли алгоритм працює найдовше.
- 3) Порівнювати з основними функціями: порівнюйте складність вашого алгоритму з відомими функціями, такими як $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$ тощо, щоб зрозуміти, наскільки ефективним є ваш алгоритм.
- 4) Використовувати для порівняння алгоритмів: O-нотація дозволяє порівнювати ефективність різних алгоритмів, які вирішують одну й ту саму задачу.

4. Що означають вирази $O(1)$, $O(n)$, $O(n^2)$ в контексті асимптотичної складності?

Визначають, як час виконання алгоритму або використання ним пам'яті залежить від розміру вхідних даних (n). $O(1)$ — це константна складність (час не залежить від n), $O(n)$ — це лінійна складність (час пропорційний n), а $O(n^2)$ — це квадратична складність (час пропорційний квадрату n).

5. Яким чином визначити асимптотичну складність алгоритму за його кодом або математичним виразом?

Визначення асимптотичної складності (O , Θ , Ω) завжди базується на аналізі того, як швидко зростає кількість операцій або ресурсів алгоритму залежно від розміру вхідних даних n . Якщо заданий математичний вираз: Правило домінуючого члена: (у виразі вибираємо найшвидше зростаючий доданок (константи та менші степені відкидаються)), Множники-константи не враховуються.

Якщо даний код алгоритму: Подивитися на послідовні інструкції, Аналіз циклів, Розгалуження (if/else), Рекурсія, Вибрати домінуючу частину.